

Hotel Booking Cancellation Prediction: Machine Learning Pipeline Report

Team Members:

- Abdallah Hassan (ID: 2023/04630)
- Aly Mohamed (ID: 2023/05500)
- Ahmed Hamed (ID: 2023/07596)
- Moustafa Mohamed (ID: 2023/06727)
- Ahmed Ayman (ID: 2023/03485)

Course: Artificial Intelligence

Executive Summary

This report documents a comprehensive machine learning pipeline developed to predict hotel booking cancellations using the Hotel Booking Demand dataset (119,288 records, 44 features after preprocessing). The project applies Genetic Algorithm (GA) feature selection independently for each model, builds three classification models (K-Nearest Neighbors, Decision Tree, Neural Network), and compares their performance on standardized test data.

Key Finding: The Decision Tree + GA model achieved the best test F1-score of 0.7612 using 28 GA-selected features, outperforming KNN (F1=0.6931) and MLP (F1=0.7424) while maintaining strong recall (0.7450) to catch cancellations. This validates the approach of running GA separately for each estimator, as each algorithm benefits from its own optimized feature subset.

1. Introduction

1.1 Problem Statement

Hotel booking cancellations represent a critical operational challenge. Accurate predictions enable:

- Optimized inventory and overbooking management
- Improved revenue forecasting
- Targeted customer retention strategies
- Reduced financial impact from no-shows

1.2 Objective

Build and compare three classification models using model-specific Genetic Algorithm-based feature selection to predict `is_canceled` (binary: 0 = not canceled, 1 = canceled). Each model receives its own optimized feature set based on wrapper-based feature selection principles [1][2].

1.3 Dataset Overview

- **Source:** Hotel Booking Demand (public dataset)
 - **Initial Records:** 119,390 bookings
 - **Initial Features:** 33
 - **After Preprocessing:** 119,288 records, 44 features
 - **Target:** is_canceled (binary classification)
 - **Class Distribution:** ~63% not canceled, ~37% canceled (slight imbalance)
 - **Train/Val/Test Split:** 60% / 20% / 20% (stratified)
-

2. Data Preprocessing & Feature Engineering

2.1 Data Cleaning

Columns Removed (leakage/sparsity):

- company (94.31% missing) -- dropped
- agent (13.69% missing) -- filled with 0

Missing Value Handling:

- country: Filled with mode (most frequent country)
- children: Filled with 0 (no children), converted to int64

Duplicate Handling:

- Removed exact duplicates post-preprocessing
- Shape before: (119,390, 29) -> Shape after: (119,327, 29) -> Final after feature engineering: (119,288, 44)

2.2 Outlier Treatment

Applied IQR-based winsorization (capping) on numeric columns:

- lead_time, adr, stays_in_week_nights, stays_in_weekend_nights, adults, children, babies
- Extreme values capped at $Q1 - 1.5 \times IQR$ and $Q3 + 1.5 \times IQR$

Result: Reduced extreme outliers while preserving data variance for legitimate extreme bookings.

2.3 Feature Engineering

Derived Features (6 new):

- **arrival_dayofweek:** Day of week from constructed arrival date
- **arrival_month_num:** Month as integer (1=January, 12=December)
- **total_stay:** Sum of weekend + weekday nights (later dropped -- highly correlated)
- **total_guests:** Sum of adults + children + babies
- **total_previous_bookings:** Sum of previous cancellations + confirmed bookings
- **got_reserved_room:** Binary (1 if reserved room type = assigned room type)

Frequency Encoding (2 new):

- hotel_freq: Proportion of bookings for each hotel type in training data
- country_freq: Proportion of bookings from each country in training data

Encoding Strategies:

- One-Hot Encoding: city column -- results in 15 city dummy variables
- Label Encoding: meal, market_segment, deposit_type, customer_type, distribution_channel, reserved_room_type, assigned_room_type, arrival_date_month

Multicollinearity Removal:

- Highly correlated columns identified: ['total_stay'] -- dropped
- Final dataset shape: (119,288, 44)

2.4 Train/Validation/Test Split

Stratified split to preserve class distribution:

- **Training (60%):** used for GA and model training
- **Validation (20%):** used for hyperparameter tuning
- **Test (20%):** held out for final evaluation only
- **Stratification:** Applied stratify=y to maintain ~63:37 ratio across splits

SMOTE Application (Training Only):

- Applied Synthetic Minority Over-Sampling Technique to training set after split
- Before SMOTE: Counter({0: 45,089, 1: 26,483})
- After SMOTE: Counter({0: 45,089, 1: 45,089}) -- perfectly balanced 1:1 ratio
- Validation and test sets: NOT resampled (preserved true distribution)
- Val: Counter({0: 15,030, 1: 8,828}) | Test: Counter({0: 15,030, 1: 8,828})

3. Feature Selection using Genetic Algorithm

3.1 GA Methodology & Design Rationale

Genetic Algorithm Feature Selection (GAFeatureSelectionCV) is a wrapper-based feature selection method that evaluates different feature subsets using an evolutionary algorithm and cross-validation scoring [2][3]. It is multi-objective: maximize CV F1-score while minimizing the number of selected features for simplicity and interpretability.

Why GA Per Model (Not One GA for All)?

Wrapper-based feature selection is model-dependent. The optimal feature subset for a Decision Tree differs from KNN or MLP because:

- Each algorithm learns patterns differently (Decision Tree: splits; KNN: distance; MLP: non-linear)
- Best subset for one model is not guaranteed optimal for others [2][3]
- Running separate GA per model gives each algorithm its best chance to perform [1]

GA Configuration:

Model	Generations	CV Folds	Scoring	Features Selected
K-Nearest Neighbors	6	3	F1	28
Decision Tree	9	3	F1	28
Neural Network (MLP)	9	3	F1	25

Process Flow:

1. GA initializes random feature subsets (population)

2. Each subset evaluated with 3-fold stratified cross-validation on training data
3. F1-score computed; best-performing subsets retained
4. Evolutionary operations (crossover, mutation) generate new subsets
5. Repeat for N generations
6. Return best subset found (most consistent high F1, smallest feature count)

3.2 GA Selected Features by Model

KNN: 28 Features Selected

```
lead_time, stays_in_weekend_nights, stays_in_week_nights, adults, children,
market_segment, is_repeated_guest, previous_cancellations, booking_changes,
deposit_type, days_in_waiting_list, customer_type, required_car_parking_spaces,
total_of_special_requests, hotel_freq, country_freq, city_Ahmedabad, city_Bangalore,
city_Bhopal, city_Chandigarh, city_Chennai, city_Delhi, city_Hyderabad, city_Indore,
city_Kolkata, city_Mumbai, got_reserved_room, total_guests
```

Insight: KNN selected behavioral and location features including booking changes, deposit type, and multiple city indicators.

Decision Tree: 28 Features Selected

```
lead_time, stays_in_week_nights, adults, children, babies, market_segment,
distribution_channel, is_repeated_guest, deposit_type, customer_type, adr,
required_car_parking_spaces, total_of_special_requests, hotel_freq, country_freq,
city_Ahmedabad, city_Bangalore, city_Chandigarh, city_Goa, city_Hyderabad,
city_Jaipur, city_Kochi, city_Kolkata, city_Lucknow, city_Mumbai, city_Pune,
got_reserved_room, total_previous_bookings
```

Insight: DT selected the broadest feature set including direct booking metrics (lead_time, adr) and multiple geographic details.

MLP: 25 Features Selected

```
lead_time, arrival_date_day_of_month, stays_in_weekend_nights, babies, meal,
market_segment, distribution_channel, is_repeated_guest,
previous_bookings_not_canceled, days_in_waiting_list, customer_type,
required_car_parking_spaces, total_of_special_requests, arrival_month_num,
country_freq, city_Bangalore, city_Chandigarh, city_Chennai, city_Goa,
city_Hyderabad, city_Kolkata, got_reserved_room, arrival_dayofweek,
total_guests, total_previous_bookings
```

Insight: MLP selected a balanced set emphasizing temporal features, behavioral metrics, and frequency-based aggregates.

3.3 Feature Importance Summary Across Models

Feature	KNN	DT	MLP	Consensus
lead_time	v	v	v	All 3
market_segment	v	v	v	All 3
total_guests	v	v	v	All 3
total_of_special_requests	v	v	v	All 3
country_freq	v	v	v	All 3

got_reserved_room	v	v	v	All 3
deposit_type	v	v	--	KNN + DT
is_repeated_guest	v	v	v	All 3
days_in_waiting_list	v	--	v	KNN + MLP
adr	--	v	--	DT only

Universal Predictors (Selected by All 3 Models):

- **market_segment:** Customer booking channel -- strong segment-level cancellation patterns
- **total_guests:** Total number of guests -- party size strongly affects cancellation likelihood
- **total_of_special_requests:** High requests indicate commitment to the booking
- **country_freq:** Frequency of booking country -- some countries have higher cancellation rates
- **got_reserved_room:** Whether assigned room matches reserved type -- affects satisfaction & commitment
- **lead_time:** Days in advance of booking -- longer lead time correlates with higher cancellation risk
- **is_repeated_guest:** Returning guests show stronger loyalty and lower cancellation tendency

4. Model Development & Hyperparameter Tuning

4.1 Model 1: K-Nearest Neighbors (KNN)

GA-Selected Features: 28

Hyperparameter Search: k in {3, 5, 7, 9, 11}

Tuning Metric: Validation F1-Score (primary), Validation Accuracy (secondary)

Hyperparameter Tuning Results:

k	Val Accuracy	Val Precision	Val Recall	Val F1-Score
3	0.7442	0.6237	0.7782	0.6924
5	~	~	~	~
7	~	~	~	~
9	0.7442	0.6237	0.7782	0.6924
11 (best)	0.7442	0.6237	0.7782	0.6924

Selected Hyperparameters:

- n_neighbors = 9 (best validation F1 among tested values)
- metric = 'euclidean' (default distance metric)
- algorithm = 'auto' (scikit-learn optimizes automatically)

4.2 Model 2: Decision Tree Classifier

GA-Selected Features: 28

Hyperparameter Grid: max_depth x min_samples_leaf x min_samples_split

Tuning Metric: Validation F1-Score

Selected Hyperparameters:

- max_depth = 15 -- allows deeper tree; leaf/split constraints prevent overfitting
- min_samples_leaf = 10 -- each leaf must have >= 10 samples

- min_samples_split = 2 -- standard; any split with 2+ samples is allowed
- criterion = 'gini' (default impurity measure)

Rationale: max_depth=15 with min_leaf=10 provides interpretable splits while controlling overfitting.

4.3 Model 3: Neural Network (MLP Classifier)

GA-Selected Features: 25

Hyperparameter Search: hidden_layer_sizes x alpha (L2 regularization)

Tuning Metric: Validation F1-Score

Selected Hyperparameters:

- hidden_layer_sizes = (32,) -- single hidden layer with 32 neurons
- alpha = 0.0001 -- moderate L2 regularization to prevent overfitting
- solver = 'adam' -- adaptive learning rate optimizer
- max_iter = default -- standard training iterations

Rationale: (32,) architecture provides non-linear feature learning. alpha=0.0001 balances regularization strength.

5. Model Evaluation & Results

5.1 Evaluation Metrics & Definitions

For each model on the hold-out test set, computed:

- **Accuracy:** $(TP+TN)/(TP+TN+FP+FN)$ -- overall correctness
- **Precision:** $TP/(TP+FP)$ -- of predicted cancellations, how many were correct
- **Recall:** $TP/(TP+FN)$ -- of actual cancellations, how many were caught
- **F1-Score:** $2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$ -- harmonic mean (primary metric)
- **Confusion Matrix:** TP, TN, FP, FN breakdown for detailed analysis

Why F1 for Primary Metric? Hotel cancellation prediction has imbalanced classes (~63:37) and both false positives (wasted overbooking) and false negatives (missed cancellations) have costs. F1 balances both without requiring explicit cost weights.

5.2 Test Set Performance Comparison

Metric	KNN + GA	Decision Tree + GA	MLP + GA
Features Selected	28	28	25
Best k / Depth / Layers	9	max_depth=15	(32,) / 0.0001
Test Accuracy	0.7459	0.8270	0.8065
Test Precision	0.6265	0.7781	0.7317
Test Recall	0.7755	0.7450	0.7534
Test F1-Score	0.6931	0.7612	0.7424

Key Observations:

1. Decision Tree dominates F1-score (0.7612), outperforming MLP by 2.5% and KNN by 9.9%
2. MLP has highest precision (0.7317): most conservative; fewer false alarms
3. MLP has highest recall (0.7534) narrowly ahead of Decision Tree (0.7450)

4. KNN significantly underperforms on F1 despite reasonable recall -- precision is the bottleneck

5.3 Detailed Results by Model

KNN + GA (k=9, 28 Features)

Validation Phase:

- Validation Accuracy: 0.7442
- Validation Precision: 0.6237
- Validation Recall: 0.7782
- Validation F1-Score: 0.6924

Test Phase (Final Evaluation):

- Test Accuracy: 0.7459
- Test Precision: 0.6265
- Test Recall: 0.7755
- Test F1-Score: 0.6931

Confusion Matrix (Test):

	Predicted Not Cancel	Predicted Cancel	Total Actual
Actual Not Cancel	10,949	4,081	15,030
Actual Cancel	1,982	6,846	8,828
Total Predicted	12,931	10,927	23,858

Interpretation: KNN correctly identified 6,846 cancellations but generated 4,081 false positives -- low precision is the main weakness. High recall (77.5%) means most cancellations are caught.

Decision Tree + GA (max_depth=15, min_leaf=10, min_split=2, 28 Features)

Validation Phase:

- Validation Accuracy: 0.8347
- Validation Precision: 0.7851
- Validation Recall: 0.7617
- Validation F1-Score: 0.7732

Test Phase (Final Evaluation):

- Test Accuracy: 0.8270
- Test Precision: 0.7781
- Test Recall: 0.7450
- Test F1-Score: 0.7612 <- BEST

Confusion Matrix (Test):

	Predicted Not Cancel	Predicted Cancel	Total Actual
Actual Not Cancel	13,154	1,876	15,030
Actual Cancel	2,251	6,577	8,828
Total Predicted	15,405	8,453	23,858

Interpretation: Decision Tree achieves highest F1 (0.7612) with minimal validation-to-test gap (0.7732 vs 0.7612 = 1.6%), confirming good generalization. Precision of 77.8% means most flagged cancellations are real. Catches 74.5% of all actual cancellations.

MLP + GA (hidden_layers=(32,), alpha=0.0001, 25 Features)

Validation Phase:

- Validation Accuracy: 0.8110
- Validation Precision: 0.7322
- Validation Recall: 0.7714
- Validation F1-Score: 0.7513

Test Phase (Final Evaluation):

- Test Accuracy: 0.8065
- Test Precision: 0.7317
- Test Recall: 0.7534
- Test F1-Score: 0.7424

Confusion Matrix (Test):

	Predicted Not Cancel	Predicted Cancel	Total Actual
Actual Not Cancel	12,591	2,439	15,030
Actual Cancel	2,177	6,651	8,828
Total Predicted	14,768	9,090	23,858

Interpretation: MLP shows balanced precision (73.2%) and recall (75.3%). Moderate validation-to-test gap (0.7513 vs 0.7424 = 1.2%) suggests slight overfitting on validation patterns but still acceptable.

5.4 Model Comparison & Selection Summary

F1-Score Ranking:

1. Decision Tree: 0.7612 <- Selected as Best Model
2. MLP: 0.7424 (2.5% lower than DT)
3. KNN: 0.6931 (9.9% lower than DT)

Why Decision Tree is Recommended:

- **Highest F1-score (0.7612):** best balance of precision-recall
- **Strong recall (0.7450):** catches 3 out of 4 actual cancellations
- **Stable performance:** minimal validation-to-test gap (1.6%), no severe overfitting
- **Interpretable:** decision rules can be explained to stakeholders
- **Efficient dimensionality:** 28 features (out of 44), balancing complexity and information
- **Actionable insights:** feature importance easily extracted for business decisions

6. Conclusion & Recommendations

6.1 Key Findings

1. GA Feature Selection Success:

- Reduced dimensionality by 36-43% depending on model without sacrificing performance
- Improved model interpretability and reduced training time
- Identified universal predictors: market_segment, total_guests, total_of_special_requests, country_freq,

got_reserved_room, is_repeated_guest, lead_time

- Validated design choice: Running GA separately per model gave each algorithm its optimized feature subset

2. Model Performance Rankings:

- Decision Tree achieved 0.7612 F1-score, 74.5% recall -- best overall balance
- MLP provides balanced precision/recall (73.2% / 75.3%) -- useful conservative fallback
- KNN has highest recall (77.6%) but lowest precision (62.7%) -- too many false alarms

3. Feature Insights by Category:

- **Behavioral Features** (previous cancellations, special requests, repeated guest) -- highly predictive across all models
- **Geographic Features** (city dummies, country frequency) -- booking origins have distinct cancellation patterns
- **Booking Characteristics** (lead time, ADR, market segment, deposit type) -- matter most for DT and MLP
- **Temporal Patterns** (day of month, arrival week, season) -- important for MLP's non-linear learning
- **Party Composition** (total guests, adults, children) -- larger groups more committed to their booking

6.2 Business Recommendations

For Hotel Operations Team:

- Deploy Decision Tree Model for Production -- best F1 (0.7612) and strong recall
 - Use to flag high-risk bookings for proactive intervention
- Implement Threshold Tuning -- lower threshold to 0.40 to increase recall; raise to 0.60 for fewer false alarms
- Risk Stratification -- High risk ($P > 0.7$): require deposits; Medium: flexible cancellation; Low: standard terms

For Revenue Management:

- Premium Pricing & Deposit Strategies for high-cancellation segments
- Proactive Retention Incentives -- identify predicted cancellations 30+ days before arrival
- Dynamic Cancellation Policies -- flexible for low-risk, strict for high-risk bookings

6.3 Limitations & Caveats

1. **Stationarity Assumption:** Model assumes historical patterns persist. Economic changes may shift behavior.
2. **Geographic Limitation:** Model trained on specific hotels/region. Generalization uncertain.
3. **Temporal Scope:** Data from a single period. Seasonal effects across years not fully explored.
4. **No External Features:** Lacks economic indicators, competitor pricing, local events.
5. **Fairness Not Audited:** Performance may vary across regions; bias audit recommended before production.

6.4 Future Work

1. **Ensemble Methods:** Combine all three models (voting, stacking) to improve F1 beyond 0.7612
2. **Threshold Optimization:** Use business cost matrix to find optimal decision threshold
3. **Advanced Feature Engineering:** Polynomial features (lead_time x market_segment), rolling aggregates
4. **Temporal Analysis:** Investigate day-of-week and season effects; build peak vs off-season models
5. **External Data Integration:** Incorporate weather, local events, competitor pricing
6. **Model Explainability:** Apply SHAP values for per-prediction explanations to stakeholders
7. **Fairness & Bias Audit:** Disaggregate test performance by country, market segment, hotel type

7. References

- [1] Scikit-learn development team. (2024). Feature selection. https://scikit-learn.org/stable/modules/feature_selection.html
- [2] Brainalyst Academy. (2024). Wrapper Methods: Feature Selection Algorithm. <https://brainalystacademy.com/wrapper-methods/>
- [3] Feature Engineering. (2019). Classes of Feature Selection Methodologies. <http://www.feat.engineering/classes-of-feature-selection-methodologies>
- [4] Kingma, D. P., & Ba, J. (2014). Adam: A method for stochastic optimization. arXiv preprint arXiv:1412.6980.
- [5] Goldberg, D. E. (1989). Genetic algorithms in search, optimization, and machine learning. Addison-Wesley.
- [6] Hastie, T., Tibshirani, R., & Friedman, J. (2009). The elements of statistical learning (2nd ed.). Springer.
-

Appendix: Project Execution Summary

Project Timeline:

- **Data Loading & EDA:** Exploratory analysis on 119,390 records, visualized distributions, correlations, and outliers
- **Preprocessing:** Cleaning, outlier handling (IQR capping), feature engineering (6 derived + 2 frequency features)
- **Train/Val/Test Split:** Stratified 60/20/20 split with SMOTE on training only
- **GA Feature Selection:** Separate GA runs for KNN (6 gen), DT (9 gen), MLP (9 gen)
- **Hyperparameter Tuning:** Grid search on validation set for each model
- **Test Evaluation:** Final metrics on held-out test set (23,858 records)

Computational Resources:

- Dataset size: 119,288 x 44 features (after preprocessing)
- SMOTE balanced training: 45,089 x 45,089 (class 0 vs class 1)
- Total execution time: ~45-60 minutes on standard hardware (multi-core CPU)

Model Selection Outcome:

- Best Model: Decision Tree + GA (F1=0.7612, Recall=0.7450, Accuracy=0.8270)
- Recommended for Production: Yes, with threshold tuning and ongoing performance monitoring

Next Steps for Deployment:

1. Export trained Decision Tree model to production environment
2. Implement API endpoint for real-time cancellation probability predictions
3. Set up daily retraining pipeline with new booking data
4. Monitor model performance drift monthly; retrain if F1 drops >5%
5. A/B test threshold values (0.5 vs 0.45 vs 0.40) on subset of bookings